

UNIT IV – COLLECTIONS, EXCEPTION HANDLING, AND FILE I/O

Advanced Exception Handling (User-defined exceptions, try-catch nesting), Java Collections Framework: List, Set, Map (ArrayList, LinkedList, HashSet, TreeSet, HashMap), Iterators and Enhanced for loop, Introduction to File I/O: FileReader, FileWriter, BufferedReader, BufferedWriter, Reading and writing objects using Serialization, Applications: Contact book, filebased product catalogs, grade recorders

4.1 ADVANCED EXCEPTION HANDLING

User-Defined Custom Exception in Java

Java provides us the facility to create our own exceptions by extending the Java Exception class. Creating our own Exception is known as a custom exception in Java or a user defined exception in Java. In simple words, we can say that a User-Defined Custom Exception or custom exception is creating your own exception class and throwing that exception using the "throw" keyword.

Java Custom Exception

A custom exception in Java is an exception defined by the user to handle specific application requirements. These exceptions extend either the Exception class (for checked exceptions) or the RuntimeException class (for unchecked exceptions).

Why Use Java Custom Exceptions?

We use Java custom exception,

- To represent application-specific errors.
- To add clear, descriptive error messages for better debugging.
- To encapsulate business logic errors in a meaningful way.

Types of Custom Exceptions

There are two types of custom exceptions in Java.

- Checked Exceptions: It extends the Exception class. and it must be declared in the throws clause of the method signature.

- Unchecked Exceptions: It extends the RuntimeException class.

Create a User-Defined Custom Exception

1. Create a new class that extends Exception (for checked exceptions) or RuntimeException (for unchecked exceptions).
 2. Provide constructors to initialize the exception with custom messages.
 3. Add methods to provide additional details about the exception. (this is optional)
- Example**

1: Checked Custom Exception (Real-World Scenario)

// Custom Checked Exception

```
class InvalidAgeException extends Exception {  
  
    public InvalidAgeException(String m) {  
  
        super(m);  
  
    }  
  
}
```

// Using the Custom Exception

```
public class Geeks {  
  
    public static void validate(int age)  
  
        throws InvalidAgeException {  
  
        if (age < 18) {  
  
            throw new InvalidAgeException("Age must be 18 or above.");  
  
        }  
  
        System.out.println("Valid age: " + age);  
  
    }  
  
    public static void main(String[] args) {
```

```
try {
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

```
validate(12);
```

```
} catch (InvalidAgeException e) {
```

```
System.out.println("Caught Exception: " + e.getMessage());
```

```
} }}
```

Output

Caught Exception: Age must be 18 or above.

Explanation: The above example defines a custom checked exception `InvalidAgeException` that is thrown when an age is below 18. The `validate()` method checks the age and throws the exception if the age is invalid. In the `main()` method, the exception is caught and the error message is printed.

Example 2: Unchecked Custom Exception

```
// Custom Unchecked Exception
```

```
class DivideByZeroException extends RuntimeException {
```

```
public DivideByZeroException(String m) {
```

```
super(m);
```

```
} }
```

```
// Using the Custom Exception
```

```
public class demo {
```

```
public static void divide(int a, int b) {
```

```
if (b == 0) {
```

```
throw new DivideByZeroException("Division by zero is not allowed."); }
```

```
System.out.println("Result: " + (a / b));
```

```
}  
  
public static void main(String[] args) {  
  
    try {  
  
        divide(10, 0);  
  
    } catch (DivideByZeroException e) {  
  
        System.out.println("Caught Exception: " + e.getMessage());  
  
    } } }
```

Output

Caught Exception: Division by zero is not allowed.

Explanation: The above example defines a custom unchecked exception `DivideByZeroException` that is thrown when we are trying to divide by zero. The `divide()` method checks if the denominator is zero and throws the exception if true. In the **main()** method, the exception is caught and the error message is printed.

Nested try blocks in Exception Handling in Java

In Java, we can use a try block within another try block. This is called nested try blocks. Each time a try statement is entered, the context of that exception is pushed onto a stack. If an exception occurs in the inner try block and is not caught by its corresponding catch block, the exception propagates to the outer try block. This process continues until the exception is either caught or reaches the default exception handler.

Nested try blocks are useful for handling exceptions at different levels of code. If an exception occurs in a parent try block, the control jumps directly to the matching catch block in the parent or outer try block, and any nested try blocks are skipped. If an exception occurs in an inner try block and is not caught, it propagates to the outer try block.

Basic Nested Try Blocks

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

Example 1: This example demonstrates nested try-catch blocks, where an `ArrayIndexOutOfBoundsException` is caught in the outer block, and an `ArithmeticException` is handled in the inner block.

```
class demo {  
  
    // Main method  
  
    public static void main(String args[])  
  
    { // Main try block  
  
        try {  
  
            // Initializing array  
  
            int a[] = { 1, 2, 3, 4, 5 };  
  
            // Trying to print element at index 5  
  
            System.out.println(a[5]);  
  
            // Inner try block (try-block2)  
  
            try {  
  
                // Performing division by zero  
  
                int x = a[2] / 0; // This will throw ArithmeticException  
  
            } catch (ArithmeticException e2)  
  
            {  
  
                System.out.println("Division by zero is not possible");  
  
            }  
  
        } catch (ArrayIndexOutOfBoundsException e1)  
  
        {  
  
        }  
  
    }  
  
}
```

```
System.out.println("ArrayIndexOutOfBoundsException");  
System.out.println("Element at such index does not exist"); } } }
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

Output

ArrayIndexOutOfBoundsException

Element at such index does not exist

Explanation: The outer try block attempts to access an element at index 5 of the array a[], which throws an ArrayIndexOutOfBoundsException. Since the exception occurs in the outer try block, the control immediately jumps to the corresponding catch block (catch (ArrayIndexOutOfBoundsException e1)). The inner try block is never executed because the exception in the outer block interrupts the flow. Therefore, the message "Division by zero is not possible" is not printed.

4.2 JAVA COLLECTIONS FRAMEWORKS

The Java Collections Framework (JCF) is a set of classes and interfaces within the java.util package that provides a standardized architecture for representing and manipulating collections of objects. It offers efficient, reusable data structures and algorithms, allowing developers to manage groups of objects without needing to implement their own data management logic from scratch.

Key components of the JCF:

Interfaces: These define the contract for different types of collections. The main interfaces include:

Collection: The root interface for all collection types (except Map). It defines basic operations like adding, removing, and iterating over elements.

List: An ordered collection (sequence) that allows duplicate elements. Elements can be accessed by their integer index. Examples: ArrayList, LinkedList.

Set: A collection that does not allow duplicate elements. It models the mathematical set abstraction. Examples: HashSet, LinkedHashSet, TreeSet.

Queue: A collection designed for holding elements prior to processing. It typically follows a First-In, First-Out (FIFO) order. Examples: PriorityQueue, ArrayDeque.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

Map: An object that maps keys to values. It does not extend Collection because it stores key value pairs rather than single elements. Examples: HashMap, LinkedHashMap, TreeMap.

Implementations: Concrete classes that provide implementations for the collection interfaces. These offer various performance characteristics and features. For example, ArrayList uses a dynamic array for fast random access, while LinkedList uses a doubly-linked list for efficient insertions and deletions.

Algorithms: Static utility methods provided by classes like Collections (for collections) and Arrays (for arrays) that offer common operations such as sorting, searching, shuffling, and reversing elements within a collection. List Interface and Its Implementations (ArrayList, LinkedList)

4.2.1 LIST

In the Java world, a List is a collection that stores elements in a linear sequence, allowing for easy access, modification, and insertion of elements at any position.

There are two main implementations of the List interface: ArrayList and LinkedList. Let's take a closer look at both of these salty sea dogs!

a) ArrayList

ArrayLists are like this - they use an array to store elements, and when you add or remove elements, the ArrayList automatically resizes.

Here are some code snippets to create an ArrayList and add some pirate elements:

```
pirateCrew.add("Captain");
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

b) LinkedList

A LinkedList be a collection that stores elements as nodes, with each node pointing to the next node in the sequence.

Here be some code to create a LinkedList and add some pirate elements:

4.2.2 SETS

Set Interface and Its Implementations (HashSet, TreeSet)

In the Java realm, a Set be a collection that contains no duplicate elements. It be perfect for situations where ye need to ensure that each element be as unique as a pirate's parrot.

There be two main implementations of the Set interface that we'll be exploring today: HashSet and TreeSet. Let's weigh anchor and learn more about these two treasure troves!

a) HashSet

A HashSet be an unordered collection that uses a hash table for storage. It offers constant time, $O(1)$, performance for basic operations like add, remove, and contains, as long as the hash function disperses the elements properly.

Here be some code to create a HashSet and add some unique pirate treasures:



ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

b) TreeSet

A TreeSet is a sorted collection that uses a balanced binary search tree (usually a Red-Black tree) for storage. It offers logarithmic time, $O(\log n)$, performance for basic operations, but ensures that the elements be stored in ascending order.

Here be some code to create a TreeSet and add some unique pirate treasures:

Comparing HashSet and TreeSet

So, when should ye hoist the flag for HashSet, and when should ye set sail with TreeSet? It be a matter of order and performance, me hearties! If ye don't need your elements to be sorted and ye want fast performance, a HashSet be the perfect choice. But if ye need your collection to be ordered and ye be willing to trade a bit of speed for it, a TreeSet be your best mate.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY **24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV**

And with that, me hearties, our voyage through the Set interface and its implementations be complete. But don't drop anchor just yet, for more Java

4.2.3 MAP

Map Interface and Its Implementations (HashMap, TreeMap)

In Java, a Map be an object that maps keys to values. Each key must be unique, and it can be associated with only one value, just like each X on a treasure map can lead to only one buried treasure.

a) HashMap

A HashMap be an unordered collection that uses a hash table for storage. Like a HashSet, it offers constant time, $O(1)$, performance for basic operations such as put, get, and remove, as long as the hash function disperses the keys properly.

Here be some code to create a HashMap and store some pirate treasures with their corresponding X coordinates:



ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

b) TreeMap

A TreeMap is a sorted collection that uses a balanced binary search tree (usually a Red Black tree) for storage. Like a TreeSet, it offers logarithmic time, $O(\log n)$, performance for basic operations but ensures that the keys are stored in ascending order.

Here is some code to create a TreeMap and store some pirate treasures with their corresponding X coordinates:



ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

4.3 ITERATORS AND ENHANCED FOR LOOP

4.3.1 Iterators

An **iterator** is an object that lets you traverse a collection and access its elements one by one. It acts as a cursor, providing a standard way to iterate over different data structures. The Iterator interface has three main methods: hasNext(), next(), and remove().

Example Program using an Iterator:

```
import java.util.ArrayList;

import java.util.Iterator;

public class IteratorExample {

    public static void main(String[] args) {

        ArrayList<String> fruits = new ArrayList<>();

        fruits.add("Apple");

        fruits.add("Banana");
```

```

fruits.add("Orange");

// Get an iterator for the ArrayList

Iterator<String> iterator = fruits.iterator();

// Iterate through the elements

while (iterator.hasNext()) {

String fruit = iterator.next();

System.out.println("Processing: " + fruit);

// You can safely remove an element using the iterator's remove() method if

(fruit.equals("Banana")) {

iterator.remove();

```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

```

}

}

System.out.println("\nAfter removing 'Banana': " + fruits);

}

}

```

Output:

Processing: Apple

Processing: Banana

Processing: Orange

After removing 'Banana': [Apple, Orange]

4.3.2 Enhanced for Loop (For-Each Loop)

The **enhanced for loop** provides a simpler syntax for iterating over arrays and

collections. It's a syntactic sugar for an iterator, making code cleaner and less error-prone. It's primarily for reading elements, and you can't use it to modify the collection by adding or removing elements while iterating.

Example Program using an Enhanced for Loop:

```
import java.util.ArrayList;

public class EnhancedForLoopExample {

    public static void main(String[] args) {

        ArrayList<String> colors = new ArrayList<>();

        colors.add("Red");

        colors.add("Green");

        colors.add("Blue");

        // Use the enhanced for loop to iterate and print each element
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

```
        System.out.println("Using the enhanced for loop:"); for
        (String color : colors) {

            System.out.println(color);

        }

        System.out.println("\nUsing enhanced for loop with an array:");

        int[] numbers = {10, 20, 30, 40, 50};

        for (int number : numbers) {

            System.out.println(number);

        }

    }

}
```

Output:

Using the enhanced for loop:

Red

Green

Blue

Using enhanced for loop with an array:

10

20

30

40

50

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

4.4 FILE I/O (INPUT/OUTPUT)

File I/O (Input/Output) in Java involves reading data from and writing data to files. When dealing with text files, the `FileReader` and `FileWriter` classes are fundamental for character-based operations. They are part of Java's character stream hierarchy, designed for handling text data efficiently.

1. FileReader:**• Purpose:**

`FileReader` is used to read character data from a file. It reads data one character at a time.

• Constructors:

- `FileReader(String fileName)`: Creates a `FileReader` object to read from the file specified by the given path.

- `FileReader(File file)`: Creates a `FileReader` object to read from the specified `File` object.

- **Key Method:**

- `int read()`: Reads a single character and returns its integer representation (0-65535). Returns -1 if the end of the stream has been reached.

- **Usage:**

Typically, a `FileReader` is used within a loop, continuously calling `read()` until -1 is returned, indicating the end of the file. The returned integer value needs to be cast to a `char` to get the actual character.

Example of FileReader:

```
import java.io.FileReader;
import java.io.IOException;
```

```
public class ReadFile {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("example.txt")) {
            int character;
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

```
        while ((character = reader.read()) != -1) {
            System.out.print((char) character);
        }
    } catch (IOException e) {
        System.err.println("Error reading file: " + e.getMessage());
    }
}
```

2. FileWriter:

- **Purpose:**

`FileWriter` is used to write character data to a file. It writes data one character or a string at a

time.

- **Constructors:**

- `FileWriter(String fileName)`: Creates a `FileWriter` object to write to the file specified by the given path. By default, it overwrites the file if it exists. •

- `FileWriter(String fileName, boolean append)`: Creates

- a `FileWriter` object. If `append` is `true`, data is appended to the end of the file; otherwise, the file is overwritten.

- `FileWriter(File file)`: Creates a `FileWriter` object to write to the specified `File` object.

- `FileWriter(File file, boolean append)`: Similar to the above, but allows specifying append mode.

- **Key Methods:**

- `void write(int c)`: Writes a single character.

- `void write(char[] cbuf)`: Writes an array of characters.

- `void write(String str)`: Writes a string.

- **Usage:**

Data is written using the `write()` methods. It's crucial to close the `FileWriter` after writing to ensure all buffered data is flushed to the file and resources are released. `try-with resources` automatically handles closing.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

Example of FileWriter:

```
import java.io.FileWriter;
```

```
import java.io.IOException;
```

```
public class WriteFile {
```

```
    public static void main(String[] args) {
```

```
        String content = "Hello, World!\nThis is a test.";
```

```
        try (FileWriter writer = new FileWriter("output.txt")) {
```

```
            writer.write(content);
```

```
        } catch (IOException e) {
```

```
            System.err.println("Error writing to file: " + e.getMessage());
```

```
}  
}  
}
```

3) **BufferedReader**

BufferedReader is a Java class that provides a more efficient way to read text from a character input stream, like a file. It buffers the characters from the input stream, reading a larger chunk of data from the disk at once instead of one character at a time. This reduces the number of disk access operations, which are expensive, and significantly improves I/O performance.

Key Features:

- **Performance:** It's much faster for reading large amounts of text data compared to reading one character at a time.
- **readLine() Method:** This is its most notable feature. It reads a line of text, including the newline character, and returns the line as a String. This is incredibly useful for processing text files line by line.

When to use it:

- Reading a text file line by line.
- Parsing large text data, such as log files or CSVs.
- When performance is a priority.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

Example:

```
import java.io.BufferedReader;  
  
import java.io.FileReader;  
  
import java.io.IOException;  
  
public class BufferedReaderExample {  
  
    public static void main(String[] args) {
```

```

try (BufferedReader br = new BufferedReader(new FileReader("sample.txt"))) {

String line;

while ((line = br.readLine()) != null) {

System.out.println(line);

}

} catch (IOException e) {

e.printStackTrace();

} } }

```

4) **BufferedWriter**

BufferedWriter is a Java class that provides a more efficient way to write text to a character-output stream, like a file. Similar to **BufferedReader**, it uses an internal buffer to write data. Instead of writing each character to the disk immediately, it stores the characters in memory and writes them to the file in a single, larger operation when the buffer is full or when the stream is flushed.

Key Features:

- **Performance:** It minimizes disk writes, which boosts performance.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY **24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV**

- **newLine() Method:** This method writes a platform-specific line separator to the output stream, making your code more portable.
- **flush() Method:** This method forces any buffered data to be written to the underlying stream. It's crucial to call `flush()` or close the stream to ensure all data is written to the file.

When to use it:

- Writing large amounts of text data to a file.

- When you need to write data line by line to a file.
- When performance is important.

Example:

```
import java.io.BufferedWriter;

import java.io.FileWriter;

import java.io.IOException;

public class BufferedWriterExample {

    public static void main(String[] args) {

        try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {

            bw.write("Hello, this is a test line.");

            bw.newLine();

            bw.write("This is a new line with BufferedWriter.");

        } catch (IOException e) {

            e.printStackTrace();

        }
    }
}
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

4.5 READING AND WRITING OBJECTS USING SERIALIZATION

Serialization and Deserialization

Serialization is the process of converting an object's state into a byte stream. Think of it as **packing an object into a box** for storage or transport. This allows you to save the object to a file, send it over a network, or store it in a database.

Deserialization is the opposite process: **unpacking the object from the box** to recreate it in memory.

For a class to be serializable, it must implement the **java.io.Serializable** interface. This is a special marker interface that tells the Java Virtual Machine (JVM) the object can be converted into a byte stream. It has no methods to implement.

Writing an Object (Serialization)

To save an object to a file, you use an **ObjectOutputStream**. This stream converts your object into a byte stream and writes it to a file.

1. **Create a FileOutputStream:** This connects your program to the file you want to write to.
2. **Create an ObjectOutputStream:** This wraps the FileOutputStream and handles the serialization logic.
3. **Call writeObject():** Pass the object you want to save to this method.

Reading an Object (Deserialization)

To load a saved object from a file, you use an **ObjectInputStream**. This stream reads the byte stream from the file and recreates the object.

1. **Create a FileInputStream:** This connects your program to the file you want to read from.
2. **Create an ObjectInputStream:** This wraps the FileInputStream and handles the deserialization logic.

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

3. **Call readObject():** This method reads the byte stream and returns a generic Object type. You must **cast** it back to its original class.

Simple Example Program

This program demonstrates how to save a Student object and then load it back.

The Student class:

```
import java.io.Serializable;

class Student implements Serializable {

    private String name;

    private int rollNumber;

    public Student(String name, int rollNumber) {

        this.name = name;

        this.rollNumber = rollNumber;

    }

    public String toString() {

        return "Name: " + this.name + ", Roll Number: " + this.rollNumber; }

    }
```

Main Program:

```
import java.io.*;

public class StudentSerializer {

    public static void main(String[] args) {

        // --- STEP 1: Serialize and write the object ---

        Student student1 = new Student("Alice", 101);
```

ROHINI COLLEGE OF ENGINEERING AND TECHNOLOGY
24CS302 –OBJECT ORIENTED PROGRAMMING USING JAVA UNIT IV

```
String filename = "student.ser";

try (FileOutputStream fileOut = new FileOutputStream(filename);
```

```

ObjectOutputStream out = new ObjectOutputStream(fileOut)) {

out.writeObject(student1);

System.out.println("Student object saved to " + filename); }

catch (IOException e) {

e.printStackTrace();

}

// --- STEP 2: Read and deserialize the object ---

Student student2 = null;

try (FileInputStream fileIn = new FileInputStream(filename);

ObjectInputStream in = new ObjectInputStream(fileIn)) {

student2 = (Student) in.readObject();

System.out.println("\nStudent object loaded successfully:");

System.out.println(student2);

} catch (Exception e) {

e.printStackTrace();

}

}

}

```